项目详细介绍

写在前面

这份文档面向**对零知识证明和分布式系统不太熟悉的读者**，用尽量通俗的语言解释本项目在做什么、为什么要做、怎么做的、做到了什么程度。如果你是第一次接触这个项目，建议从头到尾读一遍。

一、项目在解决什么问题？

1.1 一个现实困境：推理太贵，一台机器算不过来

现在的深度学习模型越来越大。以大语言模型为例，GPT-4 级别的模型参数量达到万亿级，单次推理就需要大量的计算资源。很多场景下，一台机器根本跑不动，或者延迟太高、吞吐量太低。

分布式推理是一个自然的解决方案：把一个大模型切成好几块，分给多台机器各算一部分，然后把结果串起来。就像一条流水线：

```
输入数据 → 机器1算第1段 → 机器2算第2段 → ... → 机器N算最后一段 → 最终结果
```

1.2 新的问题：我怎么知道每台机器算对了？

分布式推理解决了算力问题，但引入了一个新问题：**信任问题**。

如果其中某台机器出了故障，或者被恶意攻击者控制，它可能返回错误的结果。更糟糕的是，你可能根本发现不了——因为你看到的只是最终输出，你不知道中间某一步是不是被篡改了。

这就像一条生产线上，你只检查了最终产品，但中间某个工序偷偷用了劣质材料，最终产品表面看起来没问题，但实际上有隐患。

1.3 零知识证明：一种密码学级别的“质检报告“

零知识证明（Zero-Knowledge Proof, ZKP） 是一种密码学技术。它的核心思想是：

> 我可以向你证明“我确实按照规定的方式完成了计算“，但不需要告诉你我具体用了什么数据。

用生活中的例子来类比：假设你要验证一个人确实解出了一道数学题，零知识证明就相当于他给你一份“做题过程的密码学签章“——你可以验证这份签章是有效的（即计算确实正确），但你不需要看到他具体的演算过程。

在我们的场景中：每台机器完成自己那一段推理后，同时生成一份零知识证明。这份证明可以让中央调度节点验证“这台机器确实按照正确的模型做了推理“，而不需要重新算一遍。

1.4 本项目的目标：把三件事结合在一起

本项目的核心工作是把以下三件事结合在一个系统里：

1. **分布式推理**：把模型切成多段，分给多台机器协同完成推理
2. **零知识证明验证**：让每台机器为自己的计算生成密码学证明，使中央节点可以验证其正确性
3. **低开销优化**：不是每台机器都需要生成证明，通过聪明的策略选择部分机器生成证明，在保持安全性的同时大幅降低成本

二、系统是怎么设计的?

2.1 整体架构：Master-Worker 模式

系统由两种角色组成：

Master（主节点）：就像工厂的总控室。它负责：

- 把输入数据发给第一台 Worker
- 收集每台 Worker 的输出，传给下一台
- 验证每台 Worker 返回的结果是否正确
- 最终汇总出完整的推理结果

Worker（工作节点）：就像流水线上的工位。每个 Worker：

- 持有模型的一个切片（比如模型的第3层到第5层）
- 收到输入后执行推理
- 根据角色，可能生成零知识证明，也可能只做轻量计算
- 把结果返回给 Master

2.2 两种 Worker 角色

并不是所有 Worker 都需要生成零知识证明（因为生成证明很慢，通常需要1-5秒）。系统设计了两种角色：

Proof 节点（证明节点）：走 `/infer` 接口。

- 执行完推理后，额外生成一份 EZKL 零知识证明
- 这份证明可以被 Master 独立验证
- 相当于这个节点的计算有了“密码学背书“

Light 节点（轻量节点）：走 `/infer_light` 接口。

- 只执行推理，不生成证明
- 计算完后做一个哈希校验
- 速度快，但安全保障弱一些
- Master 可以随时对它发起“随机挑战“，要求它补交证明

2.3 模型是怎么被切开的?

以一个 8 层的全连接神经网络为例，如果配置为 4 切片：

```
原始模型：Layer1 → Layer2 → Layer3 → Layer4 → Layer5 → Layer6 → Layer7 → Layer8

切片 1：Layer1 → Layer2        → 导出为 slice_1.onnx → 交给 Worker 1
切片 2：Layer3 → Layer4        → 导出为 slice_2.onnx → 交给 Worker 2
切片 3：Layer5 → Layer6        → 导出为 slice_3.onnx → 交给 Worker 3
切片 4：Layer7 → Layer8        → 导出为 slice_4.onnx → 交给 Worker 4
```

模型用 PyTorch 定义，按层切分后分别导出为 ONNX 格式。每个 Worker 用 ONNXRuntime 加载自己那份切片进行推理。

2.4 数据是怎么流动的？

```
用户输入 x
  ↓
Master 把 x 发给 Worker 1
  Worker 1 计算 y1 = f1(x)，返回 y1（可能附带证明）
  ↓
Master 验证 y1，然后把 y1 发给 Worker 2
  Worker 2 计算 y2 = f2(y1)，返回 y2（可能附带证明）
  ↓
... 依次类推 ...
  ↓
Master 收到最终输出 yN = fN(...f2(f1(x)))
  ↓
Master 汇总验证结果，输出最终答案
```

这就是一条**流水线式的顺序推理**：每个 Worker 的输出是下一个 Worker 的输入。

三、验证机制是怎么工作的？

3.1 为什么需要多层验证？

单一的验证手段不够用。比如：

- 只做哈希校验？恶意节点可以同时伪造输出和哈希。
- 全部做零知识证明？开销太高，每个证明要花1-5秒。

所以系统设计了**三层验证 + 随机挑战**的组合机制。

3.2 第一层（L1）：哈希校验

原理：每个 Worker 返回结果时，同时返回输出的 SHA-256 哈希值。Master 拿到输出后自己重新算一遍哈希，看是否一致。

优点：几乎不增加开销（哈希运算非常快）。

局限：如果 Worker 是恶意的，它可以同时伪造输出和对应的哈希，让 Master 看起来一切正常。所以这一层只能发现“无意的故障“，不能对抗“有意的攻击“。

3.3 第二层（L2）：零知识证明

原理：proof 节点使用 EZKL 引擎，在 Halo2 算术电路中为自己的推理过程生成零知识证明。Master 本地独立验证这份证明。

关键机制——proof-bound output：

这是系统中最重要的安全设计。在开发过程中，我们发现了一个关键问题：

> 如果 Worker 生成了一份有效的证明，但返回给 Master 的输出却是篡改过的——证明是真的，输出是假的——那验证就白做了。

为了解决这个问题，我们实现了 **proof-bound output**：

- Worker 不再用自己独立计算的推理结果作为输出
- 而是直接从生成的证明中提取输出（因为证明的公开实例里包含了正确的输出值）
- Master 也独立从证明中提取输出，与 Worker 声称的输出交叉比对

这样一来：**证明通过 = 输出必然正确**。恶意 Worker 无法生成一份有效证明的同时返回篡改结果。

效果：proof 节点上的篡改不是在“事后被检测到“，而是在“源头就被预防了“。

3.4 第三层（L3）：跨节点哈希链

原理：前一个 Worker 的输出哈希（hash_out）必须等于下一个 Worker 的输入哈希（hash_in）。这样可以检测数据在节点之间传输时是否被篡改。

例子：

```
Worker 1 输出 y1，hash_out = SHA256(y1) = "abc123"
Worker 2 收到输入，hash_in = SHA256(其收到的输入) = "abc123"
→ 如果 hash_out == hash_in，说明传输没有被篡改
```

3.5 随机挑战：给 light 节点上一道“保险“

light 节点没有生成证明，安全性相对弱。为了补偿，Master 可以随机对某些 light 节点发起“挑战“（调用 `/re_prove`）：

- 要求 light 节点把之前缓存的输入数据重新走一遍完整的证明流程
- 如果 light 节点之前偷偷篡改了结果，它要么无法生成有效证明，要么证明和之前的输出对不上

这提供了**概率性威慑**：light 节点不知道自己会不会被抽查，所以不敢轻易作弊。

四、选择性验证：怎么在安全和效率之间取平衡?

4.1 核心矛盾

全部节点都做证明 → 最安全，但太慢（证明生成是主要瓶颈，占总时延约85%）。
全部节点都不做证明 → 最快，但完全没有安全保障。

4.2 Edge-Cover 策略

系统采用了一种叫做 **edge-cover（边覆盖）** 的策略：

核心规则：对于流水线上任意相邻的两个节点 (i, i+1)，至少有一个必须是 proof 节点。

附加规则：流水线的第一个和最后一个节点必须是 proof 节点（“首尾必验”）。

用一个 8 切片的例子来说明：

```
切片:   1     2     3     4     5     6     7     8
角色:  proof light proof light proof light proof proof
        ↑                                 ↑
      首必验                            尾必验
```

在这个配置下：

- 用户设置 `verify_ratio = 25%`（希望只验证 25% 的节点）
- 但 edge-cover 策略调整后，实际有 5/8 = 62.5% 的节点是 proof 节点

- 端到端时延比全部验证降低了约 41.5%

4.3 两个容易混淆的概念

- **verify_ratio（请求验证比例）**：用户设定的参数，表示“我希望大约多少比例的节点做证明“
- **actual_proof_fraction（实际 proof 覆盖率）**：经过 edge-cover 策略调整后，实际有多少比例的节点做了证明

两者通常不相等。因为 edge-cover 策略可能会“多选“一些节点做证明，以满足“任意相邻对至少一端为 proof“的约束。

五、可见性模式：数据隐私怎么处理？

系统支持三种可见性模式，用于控制推理数据在证明中的暴露程度：

5.1 all_public（全公开）

- 输入数据以明文形式出现在证明的公开实例中
- 任何人拿到证明后都可以看到输入数据
- 计算开销最低，作为基准

5.2 hashed（哈希模式）

- 输入数据不直接暴露，而是以 **Poseidon 哈希** 的形式出现在证明中
- 验证者可以确认“输入确实通过了某个哈希“，但无法反推出原始输入
- 代价：需要在零知识电路中计算 Poseidon 哈希，额外引入约 **1.6 倍**的证明时间

5.3 private（完全隐私）

- 输入数据完全不出现在证明的公开实例中
- 实现最强的隐私保护
- 有趣的是，开销与 all_public 基本持平（因为不需要额外的哈希电路计算）

六、技术实现细节

6.1 技术栈

组件	作用
Python 3.13	运行时环境
PyTorch 2.10	定义和训练模型
ONNX / ONNXRuntime	模型格式 / 推理引擎
EZKL 23.0.5	零知识证明引擎（基于 Halo2/PLONK/KZG）
FastAPI / Uvicorn	Worker 的 HTTP 服务框架

6.2 代码结构

```
distributed/
  master.py          ← Master 调度 + 三层校验 + 随机挑战 + proof-bound 验证
  worker.py          ← Worker 推理服务（/infer, /infer_light, /re_prove）

common/
  utils.py           ← EZKL 初始化、证明生成、验证、哈希工具

models/
  configurable_model.py ← 模型定义、按层切片、ONNX 导出

scripts/
  run_stage2.py          ← 主系统功能验证
  run_experiments.py     ← 基础性能实验
  run_advanced_experiments.py ← 选择性验证 + 攻击实验
  run_p2_experiment.py   ← 可见性模式对比实验
  run_p4_p6_experiment.py ← 补充评估实验
```

```
tests/
  test_core_semantics.py  ← 18 项语义回归测试
```

6.3 系统运行流程

一次完整的推理流程是这样的：

1. **启动阶段**：每个 Worker 启动时初始化 EZKL（生成设置 → 校准 → 编译电路 → 获取结构化参考字符串 → 完成 setup），这个过程只需要做一次。
2. **推理阶段**：Master 收到请求后，按顺序将输入发给流水线上的各 Worker。每个 Worker 调用 ONNXRuntime 执行推理，proof 节点额外调用 EZKL 生成证明。
3. **验证阶段**：Master 对每个 Worker 的返回执行三层验证（L1 哈希 → L2 proof 验证 → L3 哈希链），并可能对 light 节点发起随机挑战。
4. **汇总阶段**：所有验证通过后，Master 输出最终推理结果和完整的验证报告。

七、实验结果概要

7.1 基础性能

切片数	端到端时延	证明时间占比
2	2.4 秒	~85%
4	6.1 秒	~85%
8	12.0 秒	~85%

证明生成是绝对的主要瓶颈。验证很快（每个 proof 约 30-140 毫秒）。

7.2 选择性验证效果

在 8 切片配置下，把验证请求比例从 100% 降到 25%，端到端开销降低约 41.5%。

7.3 攻击处理

在当前攻击模型（响应层篡改）下，tamper、skip、random、replay 四类攻击全部被 proof-bound 机制成功预防。

7.4 可见性模式开销

hashed 模式比 all_public 慢约 1.6 倍；private 模式与 all_public 基本持平。

八、当前系统的边界与局限

诚实地说，这个系统是一个**研究原型**，不是一个可以直接部署的生产系统。以下是它的主要局限：

1. **Master 是可信的**：如果 Master 被攻破，整个验证体系就失效了。
2. **当前是单机模拟**：虽然架构上是分布式的，但实际运行在一台机器的多个进程上，还没有在真正的多机网络上验证过。
3. **模型规模很小**：当前使用的是一个只有 480 个参数的全连接网络，主要用于验证概念。
4. **攻击模型较窄**：只验证了“Worker 正确计算但篡改返回值“这一类攻击，没有覆盖恶意 prover 在电路层面伪造计算的场景。
5. **light 节点安全性有限**：依赖外部哈希和随机挑战，不是强密码学约束。

九、总结

用一句话概括这个项目：

> 我们把零知识证明技术应用到了分布式推理场景中，设计了一个“模型切片 + 流水线推理 + 分层验证 + 选择性证明“的原型系统，在保证推理可验证性的同时，通过 edge-cover 策略把证明开销降低了约 40%。

这个系统目前是一个可运行、可验证、可测量的研究原型。它验证了“分布式推理 + ZKP“这条技术路线的可行性，但距离实际部署还有很长的路要走。